

HACKER ATTACK

CODE REVIEW



12

LISTA DE FIGURAS

Figura 1 - <i>Code Review</i> (Git GUI)	4
Figura 2 - <i>Iteração Code Review</i>	5
Figura 3 - <i>Secure Coding Owasp</i>	7
Figura 4 - <i>Ipunt Validation - Secure Code Review</i>	9

EMSE

SUMÁRIO

<i>CODE REVIEW FUNDAMENTALS</i>	4
1 INTRODUÇÃO	4
2 BENEFÍCIOS DESSA PRÁTICA.....	5
3 REVISANDO CÓDIGOS	6
3.1 Melhores práticas de Code Review.....	6
4 UTILIZE CHECKLISTS PARA O CODE REVIEW.....	7
5 <i>HANDS ON</i> – OWASP SECURE CODING DOJO	9
REFERÊNCIAS.....	10

CODE REVIEW FUNDAMENTALS

1 INTRODUÇÃO

A revisão de código voltada à segurança é um processo manual ou automatizado que examina o código-fonte de uma aplicação. O objetivo desta análise é identificar quaisquer falhas ou vulnerabilidades de segurança existentes. Um *Code Review* usual procura especificamente por erros de lógica, examina a implementação de especificações, verifica as diretrizes de estilo de programação, entre outras atividades.

Já a revisão automatizada de código é um processo no qual uma ferramenta analisa automaticamente o código-fonte de uma aplicação, usando um conjunto predefinido de regras para procurar código que apresente um problema relevante. A revisão automatizada pode encontrar problemas no código-fonte mais rapidamente do que identificá-los manualmente.

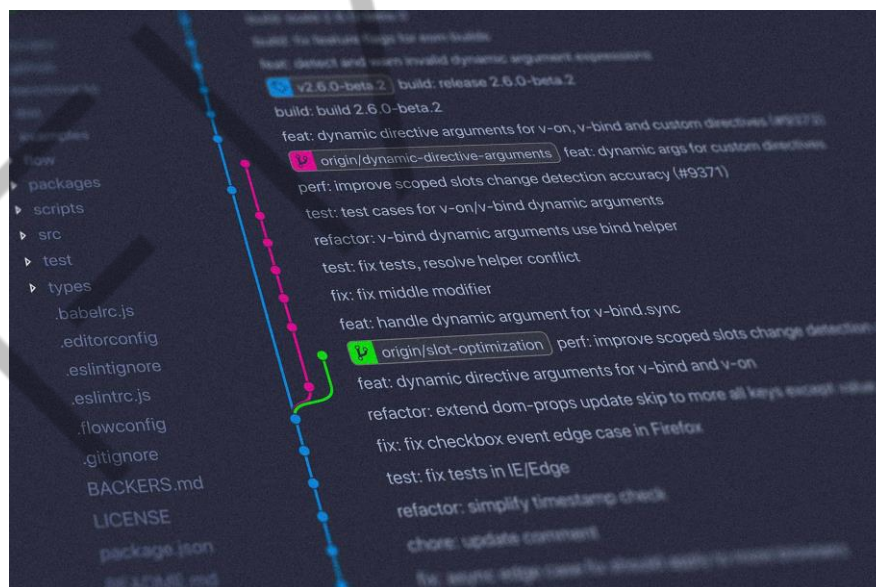


Figura 1 - Code Review (Git GUI)
Fonte: Google Imagens (2022)

As práticas atuais para que seja realizado um *Code Review* robusto e seguro envolvem o uso de revisões manuais e automatizadas juntas. Essa abordagem em conjunto captura os problemas mais potenciais. É importante ressaltar que, neste capítulo, vamos frisar principalmente a revisão de código sob a ótica de segurança.

2 BENEFÍCIOS DESSA PRÁTICA

O *Code Review* manual envolve um humano examinando o código-fonte para encontrar vulnerabilidades. A revisão manual do código ajuda a esclarecer o contexto das decisões de codificação, já as ferramentas automatizadas são mais rápidas, mas podem não levar em consideração regras de negócio e as intenções do desenvolvedor por trás da lógica geral da aplicação. Por essa razão, uma análise manual é mais estratégica e aborda questões específicas.

Como seres humanos, não apenas desenvolvedores e engenheiros, porém, todas as partes interessadas envolvidas no fluxo de trabalho de desenvolvimento podem cometer erros. É por isso que a revisão de código é uma prática essencial da indústria, que vem ganhando espaço quando o objetivo é trazer segurança ao software.



Figura 2 - Iteração *Code Review*
Fonte: Google Imagens (2022)

Em resumo, o *Code Review* é o processo pelo qual o código-fonte é avaliado por um ou mais integrante do time de desenvolvimento, que tendo o domínio da aplicação assume o papel de revisor minucioso, que buscará inconsistências, erros, possíveis bugs, problemas de design, defeitos de arquitetura e problemas de segurança. Entre os benefícios dessa prática, compõe-se o aumento da colaboração, mitigação de riscos e diminuição dos silos conhecidos.

De acordo com a Microsoft, as revisões de código também ajudam a garantir a “manutenção de longo prazo” do código e permitem que as equipes “se comuniquem sobre uma visão compartilhada de um artefato em evolução”.

Além disso, a pessoa que realiza esse processo aprende muito sobre o comportamento do software. Fazendo uma crítica bem elaborada sobre pontos específicos do código e respondendo a questões como: Por que este código está bem escrito? Por que foi feito desta forma? Isso poderia ter sido escrito de forma diferente? etc. Este é um dos benefícios de se voluntariar para revisar o código por meio da própria contribuição em projetos de código aberto.

3 REVISANDO CÓDIGOS

Existem dois tipos de revisão de código: revisões formais de código e revisões informais de código. As revisões formais do código são abrangentes. Eles seguem listas de verificação e são mais usados para software de missão crítica. As revisões informais de código são mais abertas à subjetividade, pois os adotantes preferem diretrizes e convenções do que extensas listas de verificação.

3.1 Melhores práticas de Code Review

Em um cenário de mercado atual, onde a agilidade é um fator incontestável, e as iterações tendem a ser bem curtas, as revisões formais longas e demoradas foram deixadas de lado. À medida que a velocidade de desenvolvimento aumentou, essa revisão mais longa evoluiu para um processo mais dinâmico e leve que acompanha as metodologias de desenvolvimento ágeis e modernas.

Hoje, existem ferramentas de revisão disponíveis que se integram facilmente ao SCM/IDEs. Ferramentas como o teste estáticos de segurança de aplicações (SAST) fornecem procedimentos que vão além das revisões manuais, ajudando os desenvolvedores a encontrar e corrigir vulnerabilidades. Essas ferramentas são compatíveis com vários ambientes de desenvolvimento como GitHub e GitLab ou IDEs como Eclipse e VS Code.

4 UTILIZE CHECKLISTS PARA O CODE REVIEW

Indicadores da indústria mostram que revisores que utilizam um checklist em suas revisões de código superam os que não fazem uso desse recurso.

Existem *checklists* disponíveis na internet para cada propósito de *Code Review*. Além disso, muitas dessas informações podem ser encontradas na própria documentação da linguagem/*framework*, como padrões de desenvolvimento, estilos de programação e recomendações de segurança.



Figura 3 - Secure Coding Owasp

Fonte: owasp.org (2019)

Abaixo, preparei uma *checklist* para auxiliar em revisões de código de segurança de uma aplicação web, que serve tanto para o lado cliente quanto para o lado do servidor. Apontei alguns dos pontos mais importantes para o *Code Review* de uma aplicação web moderna, tais como: validação entrada de dados, autenticação e autorização, bem como gerenciamento e criptografia de sessão.

Em suma, um exemplo checklist de segurança:

- Quais vulnerabilidades o código está suscetível?
- As entradas de dados (*inputs*) estão sendo validadas e os caracteres de escape estão sendo tratados de maneira prevenir ataques como *SQL Injection* ou *cross-site-scripting*?
- Os processos de authentication e authorization estão bem implementados?

- Dados sensíveis como dados dos usuários ou informações bancárias estão sendo tratados e armazenados de forma segura?
- As alterações realizadas no código revelam alguma informação sigilosa, como chaves de API, senhas ou nome de usuários?
- Os dados recuperados de bibliotecas ou APIs externas são adequadamente verificados?
- Os tratamentos de erros ou os registros de logs nos expõe a vulnerabilidades?
- A criptografia correta está sendo aplicada?

Definir uma *checklist* genérica, também serve para a equipe de desenvolvimento escrever código e validá-lo antes de fornecer aos revisores, *checklists* se tornam um bom termômetro para o nível de segurança que os desenvolvedores tentaram ou pensaram aplicar em seu código, considerando que a segurança era um dos requisitos da aplicação. Então, a incorporação de *checklists* em um projeto de desenvolvimento facilita o estabelecimento uma *baseline* de segurança.

Do ponto de vista da Defesa Cibernética, uma análise manual de código com foco na segurança pode auxiliar na entrega de aplicações mais seguras e robustas. Considerando que o objetivo das revisões de código seguro não é encontrar e resolver todos os possíveis problemas ou “falhas”, entretanto, fortalecer o código, tornando-o mais seguro. O intuito principal é encontrar defeitos específicos relacionados à segurança que um agente mal-intencionado poderia explorar para comprometer a tríade do **CID** (confidencialidade, integridade e disponibilidade).

Outro objetivo vital é “falhar rápido”, o que significa garantir que os bugs sejam revelados o mais cedo possível, chegando o mais próximo de sua causa raiz. Isso torna mais fácil de corrigi-las antes que elas causem graves violações de segurança após o lançamento dessa aplicação, o que poderia levar à perda de receita, multas por órgãos regulamentadores ou mesmo prejudicar a reputação da empresa.

5 HANDS ON – OWASP SECURE CODING DOJO

Agora, vamos colocar em prática, os conceitos aprendidos neste capítulo, mais especificamente o *Secure Code Review*, por intermédio de um dos diversos repositórios da OWASP disponíveis. O projeto em estudo foi criado especificamente para o aprendizado das melhores práticas de desenvolvimento seguro, que contribuem para que as pessoas tomem conhecimento das implementações que previnem as principais vulnerabilidades de aplicações na web (OWASP Top 10, 2017).

Para tanto, vamos utilizar a versão iterativa desse projeto e, por meio desse recurso, consolidar os conhecimentos que foram abordados até aqui, para isso basta acessar diretamente a Url: <https://owasp.org/SecureCodingDojo/codereview101/>



Secure Coding Dojo Code Review Categories ▾

Input Validation

Input Validation é um dos princípios básicos da segurança de software. Verificar se os valores fornecidos ao aplicativo correspondem ao tipo ou formato esperado ajuda bastante na redução da superfície de ataque. A validação é uma contramedida simples e que trás bastante resultado.

É importante observar que um erro comum é usar **block lists** para validação. Por exemplo, um aplicativo impedirá símbolos que são conhecidos por causar problemas. A fragilidade dessa abordagem é que alguns símbolos podem ser negligenciados.

Q1: Qual dos seguintes trechos de código impede uma vulnerabilidade?

```
String updateServer = request.getParameter("updateServer");
if(updateServer.indexOf(";")==-1 && updateServer.indexOf("&")==-1){
    String [] commandArgs = {
        Util.isWindows() ? "cmd" : "/bin/sh",
        "-c", "ping", updateServer
    }
    Process p = Runtime.getRuntime().exec(commandArgs);
}
```

```
String updateServer = request.getParameter("updateServer");
if(ValidationUtils.isAlphanumericOrAllowed(updateServer, "-;_[]'(){}")){
    String [] commandArgs = {
        Util.isWindows() ? "cmd" : "/bin/sh",
        "-c", "ping", updateServer
    }
    Process p = Runtime.getRuntime().exec(commandArgs);
}
```

Você está certo! O trecho de código que previne essa vulnerabilidade, é o que está usando uma listagem de permissão de entradas.

Figura 4 - Input Validation - Secure Code Review.
Fonte: [owasp.org/ SecureCodingDojo/codereview101](https://owasp.org/SecureCodingDojo/codereview101/) (2019)

REFERÊNCIAS

OWASP. **Code Review Guide**. 2017. Disponível em: https://owasp.org/www-project-code-review-guide/assets/OWASP_Code_Review_Guide_v2.pdf. Acesso em: 19 fev 2022.

THREAT INTELLIGENCE BLOG. **Secure Code Reviews: What is it, Benefits and Checklist - Code Review Guide**. 2021. Disponível em: <https://www.threatintelligence.com/blog/secure-code-reviews>. Acesso em: 22 fev 2022.

EXEMPLO